Support de cours

M1 - Dev Back

Développement Back-end & conception d'API (49h)

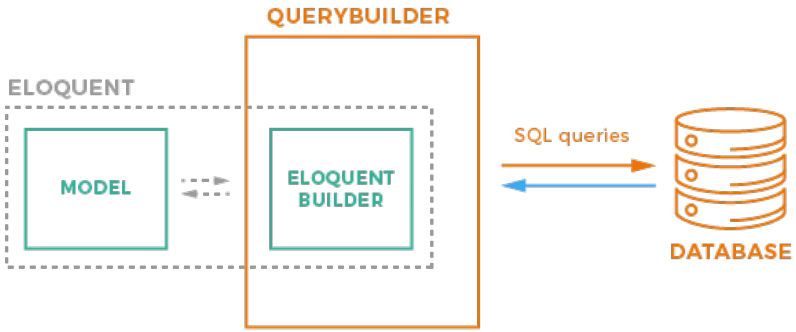
À l'issue de ce module, l'apprenant sera à même de proposer, justifier et mettre en place une solution back-end complète et des API performantes en réponse à des besoins métiers.

Date
1ère séance - 16/02/2026
2ème séance - 18/02/2026
3ème séance - 18/02/2026
4ème séance - 02/03/2026
5ème séance - 02/03/2026
6ème séance - 04/03/2026
7ème séance - 05/03/2026
8ème séance - 05/03/2026
9ème séance - 06/03/2026
10ème séance - 06/03/2026
11ème séance - 16/03/2026
12ème séance - 08/04/2026
13ème séance - 09/04/2026
14ème séance - 20/04/2026

Sommaire

- Laravel
- Serialization
  Resources
  Requests
  RGPD
  Middlewares
- Projets - Création d’une API complète

Laravel MVC



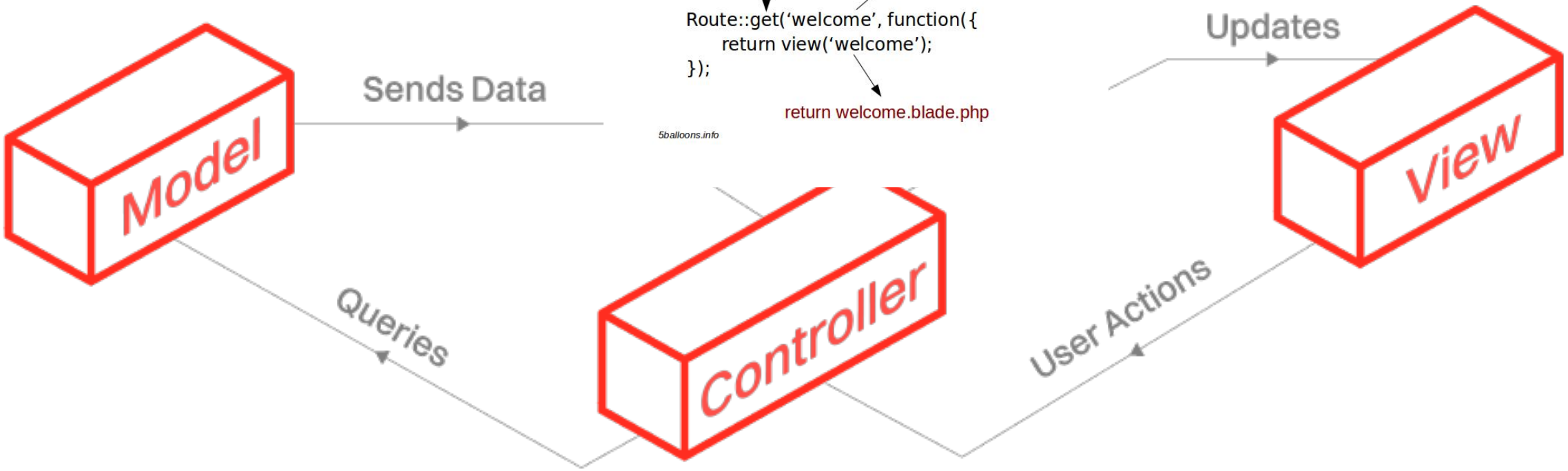
Laravel
Blade

Request Type
Unique URI

```
Route::get('welcome', function({
    return view('welcome');
});
```

return welcome.blade.php

5balloons.info



Model

Laravel fournit des modèles pour faciliter l’accès aux données de la base de données. Les modèles sont des classes PHP qui représentent une table de la base de données.

```php
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;

    protected static function newFactory(): UserFactory
    {
        return UserFactory::new();
    }

    /**
     * The attributes that are mass assignable.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'firstname',
        'lastname',
        'enterprise_id',
        'is_active',
        'email',
        'password',
        'role',
    ];

```

```php
    /**
     * The attributes that should be hidden for serialization.
     *
     * @var array<int, string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * The attributes that should be cast.
     *
     * @var array<string, string>
     */
    protected $casts = [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
        'role' => RoleEnum::class,
        'is_active' => 'boolean',
    ];

    public function enterprise(): BelongsTo
    {
        return $this->belongsTo(Enterprise::class);
    }
}
```

Migration

Les migrations dans Laravel sont un moyen de gérer le schéma de votre base de données. Elles vous permettent de créer, de modifier ou de supprimer des tables, des colonnes, des index et d'autres éléments de votre base de données.

Les migrations sont stockées dans le dossier database/migrations de votre projet Laravel. Chaque migration est un fichier PHP qui contient deux méthodes : up() et down().

La méthode up() est utilisée pour créer ou modifier les éléments de votre base de données. La méthode down() est utilisée pour supprimer ou restaurer les éléments de votre base de données.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
use Modules\Authentication\Librairies\RoleEnum;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('email')->unique();
            $table->string('firstname')->nullable();
            $table->string('lastname')->nullable();
            $table->foreignId('enterprise_id')->constrained('enterprises')->onDelete('cascade');
            $table->enum('role', RoleEnum::valuesAsArray());
            $table->boolean('is_active')->default(true);
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};

```

```php
<?php

use Illuminate\Support\Facades\Schema;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Database\Migrations\Migration;
use Modules\Authentication\Librairies\PackageEnum;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('enterprises', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('logo_url')->nullable();
            $table->enum('package', PackageEnum::valuesAsArray());
            $table->datetime('start_date')->nullable();
            $table->datetime('end_date')->nullable();
            $table->boolean('is_active')->default(true);
            $table->enum('package_n1', PackageEnum::valuesAsArray());
            $table->timestamps();
        });
    }
    ⌘L to chat, ⌘K to generate
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('enterprises');
    }
};

```

Controller

Un Controller (ou contrôleur en français) permet d'effectuer le code métier et faire le lien entre votre Model (BDD) et votre View (ou Vue en français)

Le contrôleur créé contient les méthodes suivantes :

- index() : Affiche une liste d'objets.
- show() : Affiche un objet.
- store() : Crée un nouvel objet.
- update() : Met à jour un objet existant.
- destroy() : Supprime un objet existant.

Il peut bien sur contenir une infinité de méthodes supplémentaires selon vos besoins métiers.

```php
class AuthenticatedUserController extends Controller
{
    public function __construct(protected UserService $userService) {}

    /**
     * Show authenticated user
     *
     * @return JsonResponse
     */
    public function show(): JsonResponse
    {
        return JsonApiResponse::make(
            UserResource::make(Auth::user())
        )->withSuccess()->send(200);
    }

    /**
     * Update authenticated user
     *
     * @param UserRequest $request
     * @return JsonResponse
     */
    public function update(UserRequest $request): JsonResponse
    {
        $user = $this->userService->update(
            id: Auth::user()->id,
            firstname: $request->get('firstname') ? $request->get('firstname') : null,
            lastname: $request->get('lastname') ? $request->get('lastname') : null,
        );
        return JsonApiResponse::make(UserResource::make($user))
            ->withSuccess()
            ->send(200);
    }
```

```php
    /**
     * Register
     *
     * @param RegisterRequest $request
     * @return JsonResponse
     */
    public function register(RegisterRequest $request): JsonResponse
    {
        $user = $this->userService->create(
            email: $request->get('email'),
            firstname: $request->get('firstname'),
            lastname: $request->get('lastname'),
            password: $request->get('password'),
            role: RoleEnum::tryFrom($request->get('role')),
            enterprise_id: $request->get('enterprise_id'),
        );

        return JsonApiResponse::make(UserResource::make($user))
            ->withSuccess()
            ->send(201);
    }
}
```

Routes

Les routes définissent les points d’entrée de votre API, qui sont les URL que les clients utilisent pour accéder a ressources de votre API.

Exemple

Route GET /users

La route `GET /users` renvoie une liste de tous les utilisateurs. L'action de contrôleur associée à cette route est `index()`.

Route POST /users

La route `POST /users` crée un nouvel utilisateur. L'action de contrôleur associée à cette route est `store()`.

Route GET /users/{id}

La route `GET /users/{id}` renvoie un utilisateur spécifique. L'action de contrôleur associée à cette route est `show()`.

Route PUT /users/{id}

La route `PUT /users/{id}` met à jour un utilisateur existant. L'action de contrôleur associée à cette route est `update()`.

Route DELETE /users/{id}

La route `DELETE /users/{id}` supprime un utilisateur existant. L'action de contrôleur associée à cette route est `destroy()`.

```php
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use Modules\Video\app\Http\Controllers\VideoController;
use Modules\Video\app\Http\Controllers\VideoInformationController;

/*
    |--------------------------------------------------------------------------
    | API Routes
    |--------------------------------------------------------------------------
    |
    | Here is where you can register API routes for your application. These
    | routes are loaded by the RouteServiceProvider within a group which
    | is assigned the "api" middleware group. Enjoy building your API!
    |
*/

Route::prefix('videos')->middleware(['auth:sanctum'])->group(function () {
    Route::get('enterprise/{enterpriseId}/active', VideoController::class . '@getAllActiveVideosForEnterprise');
    Route::get('enterprise/{enterpriseId}/archived', VideoController::class . '@getAllArchivedVideosForEnterprise');
    Route::get('video/{videoId}', VideoController::class . '@show');
    Route::patch('video/{videoId}/change-source', VideoController::class . '@changeSourceVideo');
    Route::patch('video/{videoId}/change-title', VideoController::class . '@changeTitleVideo');

    //Route::post('video/{videoId}/increment-views', VideoInformationController::class . '@incrementViews');

    Route::post('video', VideoController::class . '@createVideo')->middleware(['role:super_admin']);
});
```

```php

Route::prefix('player')->middleware(['verify.player.api.key'])->group(function () {
    Route::get('video/{videoId}/{hash}', VideoController::class . '@getVideoForPlayer');
    Route::post('video/{videoId}/increment-views', VideoInformationController::class . '@incrementViews');
});

Route::prefix('kpi')->middleware(['auth:sanctum'])->group(function () {
    Route::get('enterprise/{enterpriseId}', VideoController::class . '@getKPIForEnterprise');
    Route::get('enterprise/{enterpriseId}/top5', VideoController::class . '@getTop5VideosForEnterprise');
    Route::get('enterprise/{enterpriseId}/monthly-stats', VideoController::class . '@getMonthlyStatsForEnterprise');
});

```

Exemple : API Quiz